C Programming — Exercise 4.8 & 4.9 Solutions

Arrays, Strings, and File Handling — Arusha Technical College Computer Science Module

Exercise 4.8 — Arrays and Strings

Question 1: Identify and fix the error

```
#include <stdio.h>
int main() {
    int numbers[5] = {2, 4, 6, 8, 10};
    for (int i = 0; i <= 5; i++)
    {
        printf("%d ", numbers[i]);
    }
    return 0;
}
```

Error: The loop condition `i <= 5` runs from i = 0 to i = 5 (6 iterations), but the array only has valid indices 0 to 4 (5 elements). When i = 5, `numbers[5]` is out of bounds — undefined behavior (garbage value or crash).

Fix: Change `i <= 5` to `i < 5`.

```
for (int i = 0; i < 5; i++)
{
    printf("%d ", numbers[i]);
}
```

Question 2: Store five numbers in an array and calculate their sum

```
#include <stdio.h>
int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int sum = 0;

    for (int i = 0; i < 5; i++) {
        sum += numbers[i];
    }

    printf("Sum of array elements: %d\n", sum);
    return 0;
}
```

```
Sum of array elements: 150
```

Question 3: Modify the program to print the array in reverse order

Original program prints in regular order:

```
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }
```

```
    return 0;
}
```

Modified to print in reverse order:

```
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for (int i = 4; i >= 0; i--)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

```
5 4 3 2 1
```

✅ **Key change:** Start the loop counter at the last index (size − 1) and count down to 0.

Question 4: Find and fix the problem (taking a full name as input)

```
#include <stdio.h>
int main() {
    char name[20];
    printf("Enter your name: ");
    scanf("%s", name);  // Problem here
    printf("Hello, %s!\n", name);
    return 0;
}
```

Problem: `scanf("%s", ...)` stops reading at the first whitespace. If the user enters a full name like "Vincent Massawe", it only captures "Vincent" and drops "Massawe".

Fix: Use `fgets()` instead, which reads the whole line including spaces.

```
#include <stdio.h>
int main() {
    char name[20];
    printf("Enter your full name: ");
    fgets(name, sizeof(name), stdin);
    printf("Hello, %s!\n", name);
    return 0;
}
```

⚠️ **Note:** fgets() also captures the newline character (\n) at the end of input. You may want to strip it: `name[strcspn(name, "\n")] = '\0';`

Question 5: Count characters in a word without using strlen()

```
#include <stdio.h>
int main() {
    char word[50];
    int count = 0;

    printf("Enter a word: ");
    scanf("%s", word);

    while (word[count] != '\0') {
        count++;
    }
```

```
    printf("Number of characters: %d\n", count);
    return 0;
}
```

✅ **How it works:** The loop walks through the character array until it hits the null terminator '\0', counting each character — this is essentially how strlen() works internally.

Question 6: Difference between puts() and printf()

Feature	puts()	printf()
Purpose	Prints only strings	Prints formatted output (strings, integers, floats, etc.)
Format specifiers	Not supported	Supports %d, %s, %f, %c, etc.
Newline	Adds \n automatically	Does NOT add newline — must include \n yourself
Speed	Slightly faster (simpler)	Slightly slower (parses format string)
Example	puts("Hello"); → Hello + newline	printf("Hello %s\n", name);

✅ **Use case:** Use puts() for simple string output; use printf() when combining text with variables or controlling formatting.

Exercise 4.9 — File Handling

Question 1: Identify the problem

```
#include <stdio.h>
int main() {
    FILE *file;
    file = fopen("data.txt", "r");
    fprintf(file, "Hello, file handling!");
    fclose(file);
    return 0;
}
```

Problem: The file is opened in read mode ("r"), but the program tries to write to it using fprintf(). Writing is not allowed in read mode — this fails silently or causes undefined behavior.

Fix: Open the file in write mode instead: `file = fopen("data.txt", "w");`

⚠️ Also worth checking: there's no NULL check after fopen() — always good practice to include one.

Question 2: Create "info.txt" and write two lines into it

```
#include <stdio.h>
int main() {
    FILE *file = fopen("info.txt", "w");

    if (file == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    fprintf(file, "Learning C is fun!\n");
    fprintf(file, "File handling makes programming easier.\n");

    fclose(file);
```

```
    printf("Data written to info.txt successfully.\n");
    return 0;
}
```

Question 3: Read and print the contents of "message.txt"

```
#include <stdio.h>
int main() {
    FILE *file = fopen("message.txt", "r");
    char line[200];

    if (file == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    while (fgets(line, sizeof(line), file) != NULL) {
        printf("%s", line);
    }

    fclose(file);
    return 0;
}
```

✅ **Note:** fgets() reads line by line (including spaces), which is more reliable than fscanf("%s", ...) for printing a full file's contents.

Question 4: Find the error (reading one word from a file)

```
#include <stdio.h>
int main() {
    FILE *file;
    char word[50];
    file = fopen("data.txt", "r");
    fscanf(file, "%s", word);
    printf("Read: %s\n", word);
    fclose(file);
    return 0;
}
```

Error: No check whether fopen() succeeded. If "data.txt" does not exist, file will be NULL, and calling fscanf(NULL, ...) causes a crash (segmentation fault).

Fix:

```
#include <stdio.h>
int main() {
    FILE *file;
    char word[50];
    file = fopen("data.txt", "r");

    if (file == NULL) {
        printf("Error: could not open file!\n");
        return 1;
    }

    fscanf(file, "%s", word);
    printf("Read: %s\n", word);
    fclose(file);
    return 0;
}
```

Question 5: Modify to append instead of overwrite

```
#include <stdio.h>
int main() {
    FILE *file;
    file = fopen("notes.txt", "a");  // append mode
    fprintf(file, "C programming is powerful.\n");
    fclose(file);
    return 0;
}
```

✅ **Key point:** "w" mode erases existing content and creates a fresh file. "a" mode adds new content to the end without deleting what's already there.

Question 6: Count and print the number of characters in "story.txt"

```
#include <stdio.h>
int main() {
    FILE *file = fopen("story.txt", "r");
    int count = 0;
    char ch;

    if (file == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    while ((ch = fgetc(file)) != EOF) {
        count++;
    }

    printf("Number of characters: %d\n", count);
    fclose(file);
    return 0;
}
```

✅ **How it works:** fgetc() reads one character at a time until it reaches EOF (end of file), incrementing a counter each time.

Question 7: What is the problem?

```
#include <stdio.h>
int main() {
    FILE *file;
    file = fopen("notes.txt", "r");
    char text[100];
    fscanf(file, "%s", text);
    printf("Read: %s\n", text);
    return 0;
}
```

Problems:
1. No NULL check after fopen() — if the file can't be opened, fscanf(NULL, ...) will crash.
2. The file is never closed — fclose(file) is missing, which can lead to resource leaks.

Fix:

```
#include <stdio.h>
int main() {
    FILE *file;
    char text[100];

    file = fopen("notes.txt", "r");
```

```
    if (file == NULL) {
        printf("Error: could not open file!\n");
        return 1;
    }

    fscanf(file, "%s", text);
    printf("Read: %s\n", text);
    fclose(file);
    return 0;
}
```

Methods I Recommend You Start With

✅ Always check `if (file == NULL)` right after fopen() — this is the single most common bug across these exercises.

✅ Match the file mode to your intent: "r" to read, "w" to overwrite, "a" to append.

✅ Prefer fgets() over scanf("%s", ...) when reading full lines/names with spaces.

✅ Always fclose() the file when you're done with it.